

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I Harshini R N (192421175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.

- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
--------------	-----	--	--	---	---

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Scenario-based

Name: Harshini R N

Reg No: 192421175

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5117

1. Debugging & Optimization of DES / 3DES

Python

```
from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad, unpad

key = b'1234567890ABCDEF12345678' # 24-byte key
data = input("Enter plaintext: ").encode()

cipher = DES3.new(key, DES3.MODE_ECB)

ciphertext = cipher.encrypt(pad(data, 8))
print("Ciphertext:", ciphertext.hex())

plain = unpad(cipher.decrypt(ciphertext), 8)
print("Decrypted:", plain.decode())
```

Sample Input

```
Enter plaintext: HelloWorld
```

Sample Output

```
Ciphertext: 7d4f0b2baf2e9f5b2bc5c2d2b67d4e90
Decrypted: HelloWorld
```

2. Performance Analysis of AES Block Cipher Modes

Python

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(16)
iv = get_random_bytes(16)

msg = input("Enter message: ").encode()

cipher = AES.new(key, AES.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(msg, AES.block_size))

print("Ciphertext:", ciphertext.hex())

cipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = unpad(cipher.decrypt(ciphertext), AES.block_size)

print("Decrypted:", plaintext.decode())
```

Sample Input

Enter message: Network Security

Sample Output

Ciphertext: a45f2dcbab9342...
Decrypted: Network Security

3. Debugging RSA Cryptosystem

Python

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

key = RSA.generate(2048)

public_key = key.publickey()
private_key = key

cipher = PKCS1_OAEP.new(public_key)

msg = input("Enter message: ").encode()

ciphertext = cipher.encrypt(msg)
print("Encrypted:", ciphertext.hex())

decryptor = PKCS1_OAEP.new(private_key)
plaintext = decryptor.decrypt(ciphertext)

print("Decrypted:", plaintext.decode())
```

Sample Input

Enter message: Cryptography

Sample Output

Encrypted: 84ab9f4e...

Decrypted: Cryptography

4. Diffie-Hellman Key Exchange

Python

```
p = 23
g = 5

a = int(input("Enter Alice private key: "))
b = int(input("Enter Bob private key: "))

A = pow(g, a, p)
B = pow(g, b, p)

shared_alice = pow(B, a, p)
shared_bob = pow(A, b, p)

print("Alice Public Key:", A)
print("Bob Public Key:", B)
print("Shared Key (Alice):", shared_alice)
print("Shared Key (Bob):", shared_bob)
```

Sample Input

```
Enter Alice private key: 6
Enter Bob private key: 15
```

Sample Output

```
Alice Public Key: 8
Bob Public Key: 19
Shared Key (Alice): 2
Shared Key (Bob): 2
```


5. Elliptic Curve Cryptography (ECC)

```
Python

from Crypto.PublicKey import ECC

key = ECC.generate(curve='P-256')

public_key = key.public_key()

print("Private Key:")
print(key)

print("\nPublic Key:")
print(public_key)
```

Sample Output

```
Private Key:
EccKey(curve='NIST P-256', point_x=..., point_y=..., d=...)

Public Key:
EccKey(curve='NIST P-256', point_x=..., point_y=...)
```